BÀI TẬP LÝ THUYẾT
LẬP KẾ HOẠCH VÀ ĐỒ THỊ KẾ HOẠCH TRONG AI

Họ và tên	Lê Minh Hiếu
MSSV / Lớp	3124720023 / DNT1241

Mục tiêu học tập

- Mô tả bài toán lập kế hoạch cổ điển bằng trạng thái, mục tiêu, toán tử và các tập Precondition/Add/Delete.
- Xây dựng planning graph theo GraphPlan với các mức trạng thái S_i, mức hành động A_i và các hành động No-op.
- Nhận diện và giải thích các quan hệ mutex: Inconsistent Effects, Interference và Competing Needs.
- Phân tích điều kiện dừng của GraphPlan và trích xuất kế hoạch từ planning graph.
- Vận dụng GraphPlan vào các bài toán Crane, Valet Parking và môi trường planning rời rạc.

Hướng dẫn: Trả lời bằng tiếng Việt, trình bày rõ các bước suy luận. Với câu có planning graph, có thể vẽ trực tiếp lên hình hoặc bổ sung bảng. No-op được tính là một hành động khi xét mutex.

Bài 1. Khái niệm cơ bản trong GraphPlan

Hãy định nghĩa ngắn gọn các khái niệm sau và nêu ý nghĩa của chúng trong việc xây dựng planning graph.

a) Interference (xung đột do can thiệp):

Hai hành động ở cùng một mức hành động A_i có quan hệ Interference khi hiệu ứng của hành động này xóa (phủ định) một tiền điều kiện của hành động kia. Vì một hành động phá điều kiện mà hành động kia cần, kết quả phụ thuộc thứ tự thực hiện, nên hai hành động không thể được thực hiện song song trong cùng một bước. Trong planning graph, cặp này được đánh dấu mutex để khi trích xuất kế hoạch không chọn cả hai ở cùng một mức.

b) Inconsistent Effects / Inconsistency (hiệu ứng không nhất quán):

Hai hành động ở cùng mức A_i có Inconsistent Effects khi hiệu ứng của hành động này phủ định hiệu ứng của hành động kia: một hành động thêm literal p, hành động kia xóa p (tức thêm ¬p). Nếu thực hiện cả hai cùng lúc thì trạng thái sau đó không xác định được p đúng hay sai, nên chúng mutex. Ở mức trạng thái, khái niệm tương ứng là Inconsistency: hai literal p và ¬p luôn mutex; ngoài ra hai literal mutex khi mọi cặp hành động tạo ra chúng ở mức trước đều mutex (inconsistent support). Các quan hệ này giúp planning graph loại bỏ những tổ hợp sự kiện không thể đúng đồng thời.

c) Competing Needs (tiền điều kiện cạnh tranh):

Hai hành động ở mức A_i có Competing Needs khi một tiền điều kiện của hành động này mutex với một tiền điều kiện của hành động kia ở mức trạng thái S_i. Vì các điều kiện cần cho hai hành động không thể cùng đúng ở một thời điểm, hai hành động không thể được thực hiện cùng nhau.

Quan hệ này lan truyền mutex từ mức trạng thái sang mức hành động, giúp đồ thị phản ánh chính xác hơn những gì có thể đạt được cùng lúc.

d) Có những cách nào để tìm một kế hoạch trong mô hình lập kế hoạch cổ điển?

Thứ nhất, tìm kiếm tiến trong không gian trạng thái (progression): xuất phát từ trạng thái đầu, áp dụng các toán tử thỏa tiền điều kiện cho đến khi đạt trạng thái thỏa mục tiêu; có thể dùng BFS, IDS hoặc A* với heuristic như bỏ qua delete list (FF, h_add, h_max). Thứ hai, tìm kiếm lùi (regression): xuất phát từ mục tiêu, chỉ xét các hành động liên quan và nhất quán với mục tiêu, thay mục tiêu bằng tiền điều kiện của hành động cho tới khi trạng thái đầu thỏa tập mục tiêu. Thứ ba, dùng planning graph (GraphPlan): mở rộng đồ thị theo từng mức kèm quan hệ mutex, rồi trích xuất kế hoạch bằng tìm kiếm lùi trên đồ thị. Thứ tư, mã hóa bài toán thành SAT (SATPlan) hoặc CSP với độ dài kế hoạch cố định và dùng bộ giải. Cuối cùng, lập kế hoạch trong không gian kế hoạch (partial-order planning), xây dựng kế hoạch thứ tự bộ phận bằng cách thêm hành động, liên kết nhân quả và ràng buộc thứ tự.

e) Khi dùng heuristic dựa trên GraphPlan, những giả định nào của classical planning được nới lỏng? Vì sao điều này hữu ích hơn so với tìm kiếm ngây thơ?

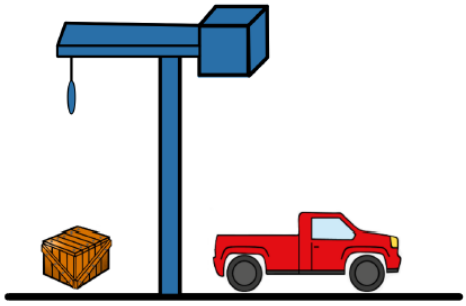
Planning graph là một bài toán nới lỏng: các literal một khi xuất hiện thì được giữ mãi nhờ No-op, nên mỗi mức trạng thái chỉ tăng chứ không mất literal (tương đương bỏ qua tác động của delete list lên khả năng đạt được); nhiều hành động được phép xảy ra song song trong cùng một mức; và mutex chỉ được xét theo từng cặp, không xét ràng buộc giữa ba hành động hay ba literal trở lên. Vì vậy mỗi mức S_i không phải là một trạng thái cụ thể mà là tập các literal có thể đúng sau i bước. Từ đó có các heuristic như mức đầu tiên một literal xuất hiện (level cost), max-level, level-sum và set-level; max-level và set-level là heuristic chấp nhận được (không ước lượng quá chi phí thật).

Điều này hữu ích hơn tìm kiếm ngây thơ vì planning graph xây được trong thời gian đa thức theo số literal và số hành động, trong khi số trạng thái có thể tăng theo hàm mũ. Heuristic cho biết khoảng cách ước lượng tới mục tiêu để A* hoặc tìm kiếm tham lam ưu tiên các trạng thái hứa hẹn, và còn phát hiện sớm trường hợp mục tiêu không thể đạt (literal mục tiêu không bao giờ xuất hiện hoặc luôn mutex), thay vì duyệt mù toàn bộ không gian trạng thái.

Bài 2. Lập kế hoạch ký hiệu - Bài toán cần cẩu

Cho một cần cẩu, một kiện hàng và một xe tải. Ban đầu kiện hàng ở bên trái, xe tải ở bên phải và cần cẩu quay về bên trái. Mục tiêu là đưa kiện hàng lên xe tải và cuối cùng cần cẩu quay về bên trái.

Cần cẩu có thể quay giữa trái và phải, có hoặc không mang kiện hàng. Cần cẩu có thể nhấc kiện hàng nếu cả hai ở cùng phía. Kiện hàng chỉ có thể được đưa lên xe tải bằng cần cẩu.



a) Xác định các biến/đối tượng, các predicate cần dùng, trạng thái ban đầu và trạng thái mục tiêu.

Các đối tượng gồm cần cẩu, kiện hàng (Box), xe tải (Truck) và hai vị trí Left (L, bên trái), Right (R, bên phải). Các predicate được dùng như sau.

Predicate	Ý nghĩa
CraneAt(x)	Cần cẩu đang quay về phía x (x ∈ {L, R})
BoxAt(x)	Kiện hàng nằm trên mặt đất ở phía x
TruckAt(x)	Xe tải ở phía x (không đổi trong bài)
Holding	Cần cẩu đang mang kiện hàng
HandEmpty	Cần cẩu không mang gì
OnTruck	Kiện hàng đã nằm trên xe tải

Trạng thái ban đầu: CraneAt(L) ∧ BoxAt(L) ∧ TruckAt(R) ∧ HandEmpty.

Trạng thái mục tiêu: OnTruck ∧ CraneAt(L).

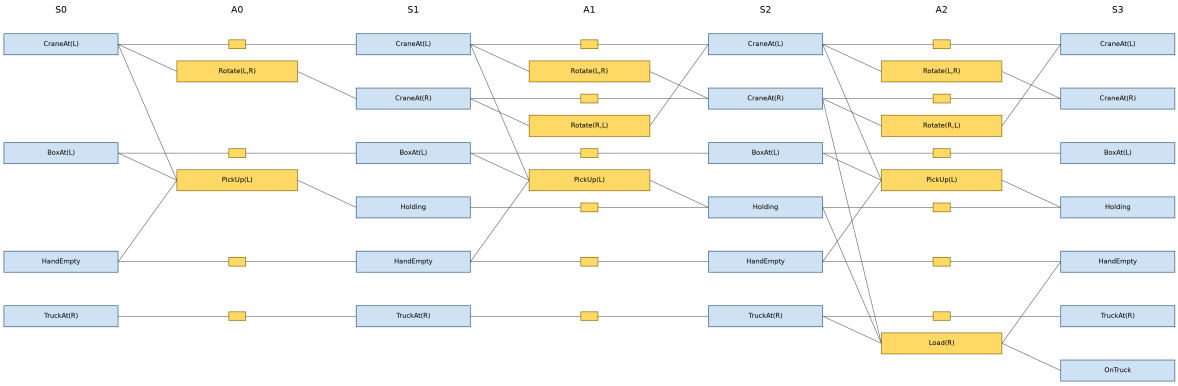
b) Xây dựng các toán tử hành động. Với mỗi toán tử, ghi rõ Preconditions, Add List và Delete List.

Toán tử	Preconditions	Add List	Delete List
Rotate(x, y) (quay từ phía x sang phía y, x ≠ y)	CraneAt(x)	CraneAt(y)	CraneAt(x)
PickUp(x) (nhấc kiện hàng ở phía x)	CraneAt(x), BoxAt(x), HandEmpty	Holding	BoxAt(x), HandEmpty
Load(x) (đặt kiện hàng lên xe tải ở phía x)	Holding, CraneAt(x), TruckAt(x)	OnTruck, HandEmpty	Holding

Khi cần cẩu quay lúc đang mang hàng thì kiện hàng đi theo cần cẩu, nên Rotate không cần cập nhật vị trí kiện hàng. Đề bài không yêu cầu đặt kiện hàng xuống đất nên không cần thêm toán tử PutDown. Các hành động cụ thể là Rotate(L, R), Rotate(R, L), PickUp(L), PickUp(R) và Load(R) (vì xe tải luôn ở bên phải). Kế hoạch ngắn nhất là PickUp(L), Rotate(L, R), Load(R), Rotate(R, L).

c) Vẽ planning graph cho 3 bước đầu tiên. Có thể dùng hình minh họa thay cho proposition nếu biểu diễn rõ ràng.

Planning graph từ S_0 đến S_3 được vẽ ở hình dưới; ô xanh là literal, ô vàng là hành động, ô vuông nhỏ là No-op, các đường nối là cạnh tiền điều kiện và cạnh add (cạnh delete không vẽ để hình đỡ rối).

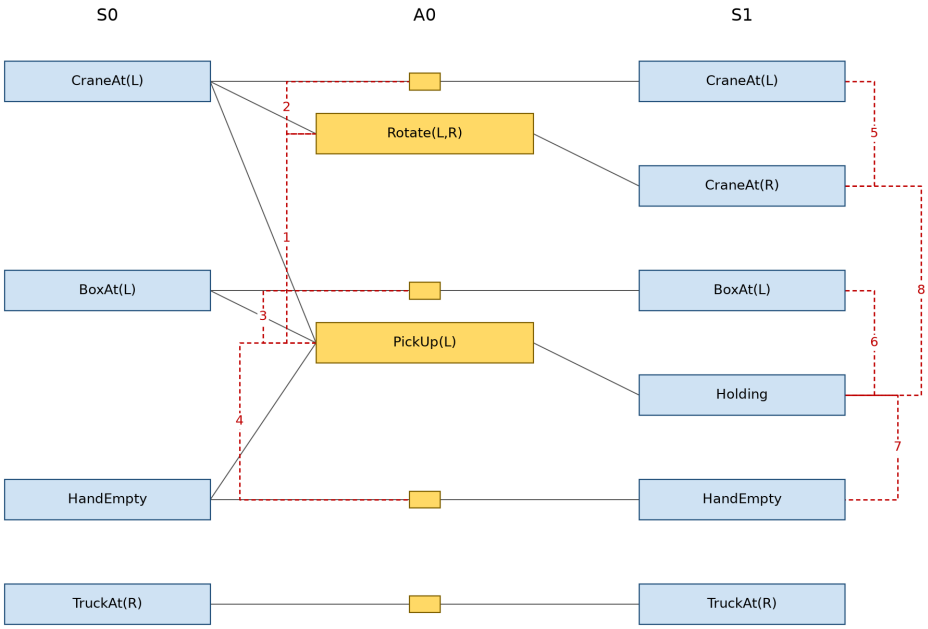


Planning graph 3 bước đầu của bài toán cần cẩu

Mức	Nội dung
S_0	CraneAt(L), BoxAt(L), HandEmpty, TruckAt(R)
A_0	PickUp(L), Rotate(L, R) và No-op của 4 literal ở S_0
S_1	CraneAt(L), CraneAt(R), BoxAt(L), Holding, HandEmpty, TruckAt(R)
A_1	PickUp(L), Rotate(L, R), Rotate(R, L) và No-op của 6 literal ở S_1. Load(R) chưa có vì Holding và CraneAt(R) mutex ở S_1
S_2	Giống S_1 (không thêm literal mới), nhưng Holding và CraneAt(R) không còn mutex vì có cặp No-op(Holding) và Rotate(L, R) ở A_1 không mutex
A_2	PickUp(L), Rotate(L, R), Rotate(R, L), Load(R) và No-op của 6 literal ở S_2
S_3	Thêm OnTruck (do Load(R)); có đủ 7 literal

Ở S_3 đã có đủ hai literal mục tiêu OnTruck và CraneAt(L), nhưng chúng mutex: OnTruck chỉ được tạo bởi Load(R), mà Load(R) mutex với No-op(CraneAt(L)) (Competing Needs vì CraneAt(R) và CraneAt(L) mutex ở S_2) và mutex với Rotate(R, L) (Interference vì Rotate(R, L) xóa CraneAt(R) là tiền điều kiện của Load(R)). Vì vậy GraphPlan phải mở rộng thêm một mức; ở S_4 mục tiêu đạt được và kế hoạch trích xuất được là PickUp(L) → Rotate(L, R) → Load(R) → Rotate(R, L).

d) Vẽ hai mức đầu tiên của GraphPlan và chỉ ra các cặp hành động mutex; với mỗi cặp, ghi loại mutex.



Hai mức đầu $S_0 \rightarrow A_0 \rightarrow S_1$; đường đứt đỏ đánh số là các cặp mutex

Số	Cặp mutex	Loại mutex	Giải thích
1	PickUp(L) – Rotate(L, R)	Interference	Rotate(L, R) xóa CraneAt(L), là tiền điều kiện của PickUp(L)
2	Rotate(L, R) – No-op(CraneAt(L))	Inconsistent Effects (đồng thời Interference)	Rotate(L, R) xóa CraneAt(L) trong khi No-op giữ CraneAt(L)
3	PickUp(L) – No-op(BoxAt(L))	Inconsistent Effects (đồng thời Interference)	PickUp(L) xóa BoxAt(L) trong khi No-op giữ BoxAt(L)
4	PickUp(L) – No-op(HandEmpty)	Inconsistent Effects (đồng thời Interference)	PickUp(L) xóa HandEmpty trong khi No-op giữ HandEmpty
5	CraneAt(L) – CraneAt(R) (literal ở S_1)	Inconsistent Support	Hai hành động tạo ra chúng (No-op và Rotate(L, R)) mutex
6	BoxAt(L) – Holding (literal ở S_1)	Inconsistent Support	No-op(BoxAt(L)) và PickUp(L) mutex
7	Holding – HandEmpty (literal ở S_1)	Inconsistent Support	PickUp(L) và No-op(HandEmpty) mutex
8	CraneAt(R) – Holding (literal ở S_1)	Inconsistent Support	Chỉ được tạo bởi Rotate(L, R) và PickUp(L), hai hành động này mutex

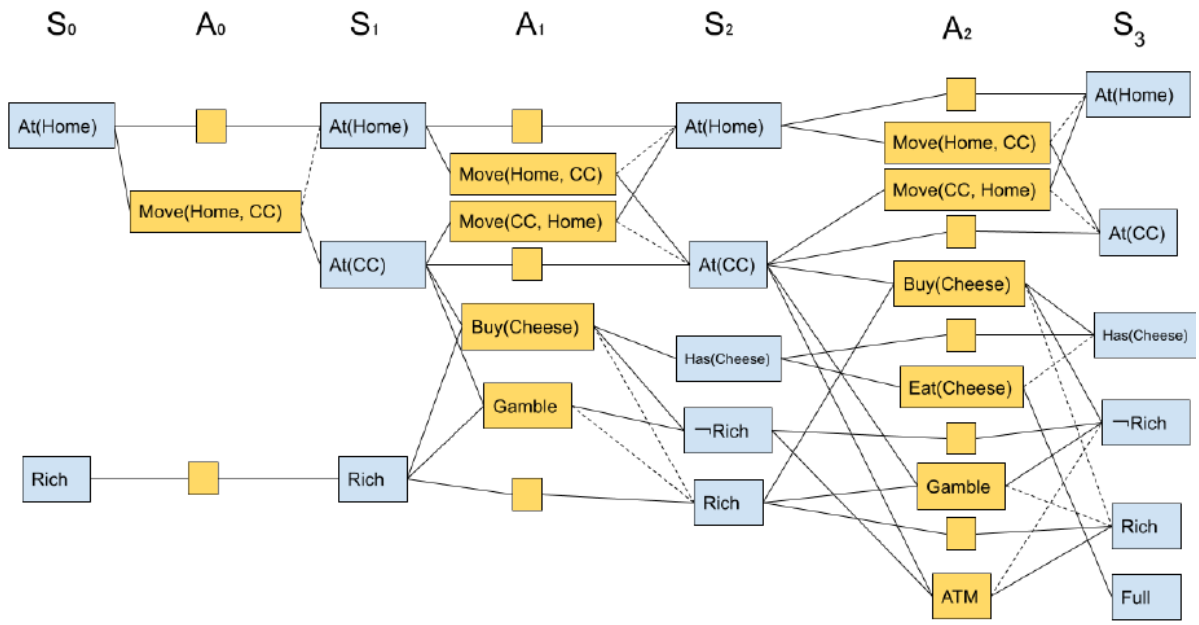
Ở A_0 không có cặp nào mutex do Competing Needs vì S_0 là trạng thái đầu, không có literal nào mutex. Các cặp hành động còn lại (ví dụ Rotate(L, R) với No-op(BoxAt(L)), hay giữa các No-op) không mutex.

Bài 3. Quan hệ Mutex trong planning graph

Pinky đi lấy thức ăn. Các hành động được cho như sau:

Hành động	**Preconditions**	**Add List**	**Delete List**
Move(A,B)	At(A)	At(B)	At(A)
Buy(Cheese)	At(ChuckyCheese), Rich	Has(Cheese), ¬Rich	
Gamble	At(ChuckyCheese), Rich	¬Rich	Rich
ATM	At(ChuckyCheese), ¬Rich	Rich	¬Rich
Eat(Cheese)	Has(Cheese)	Full	Has(Cheese)

Trạng thái bắt đầu: Rich ∧ At(Home). Mục tiêu: Full. Planning graph tương ứng được cho trong hình dưới:



Planning graph cho bài toán Pinky.

a) Ở mức A_0, hãy chỉ ra một cặp hành động mutex do Inconsistent Effects.

Cặp Move(Home, CC) và No-op(At(Home)) ở mức A_0.

b) Ở mức A_1, hãy chỉ ra một cặp hành động mutex do Interference.

Cặp Move(CC, Home) và Buy(Cheese) ở mức A_1.

c) Ở mức A_2, hãy chỉ ra một cặp hành động mutex do Competing Needs.

Cặp Eat(Cheese) và Gamble ở mức A_2.

d) Giải thích ngắn gọn vì sao mỗi cặp ở trên thỏa loại mutex tương ứng.

Câu a: Move(Home, CC) có At(Home) trong Delete List, tức làm At(Home) sai, trong khi No-op(At(Home)) giữ At(Home) đúng ở S_1. Hiệu ứng của hai hành động phủ định nhau nên đây là Inconsistent Effects.

Câu b: Move(CC, Home) xóa At(CC), mà At(CC) (At(ChuckyCheese)) là tiền điều kiện của Buy(Cheese). Hành động này phá điều kiện mà hành động kia cần nên đây là Interference.

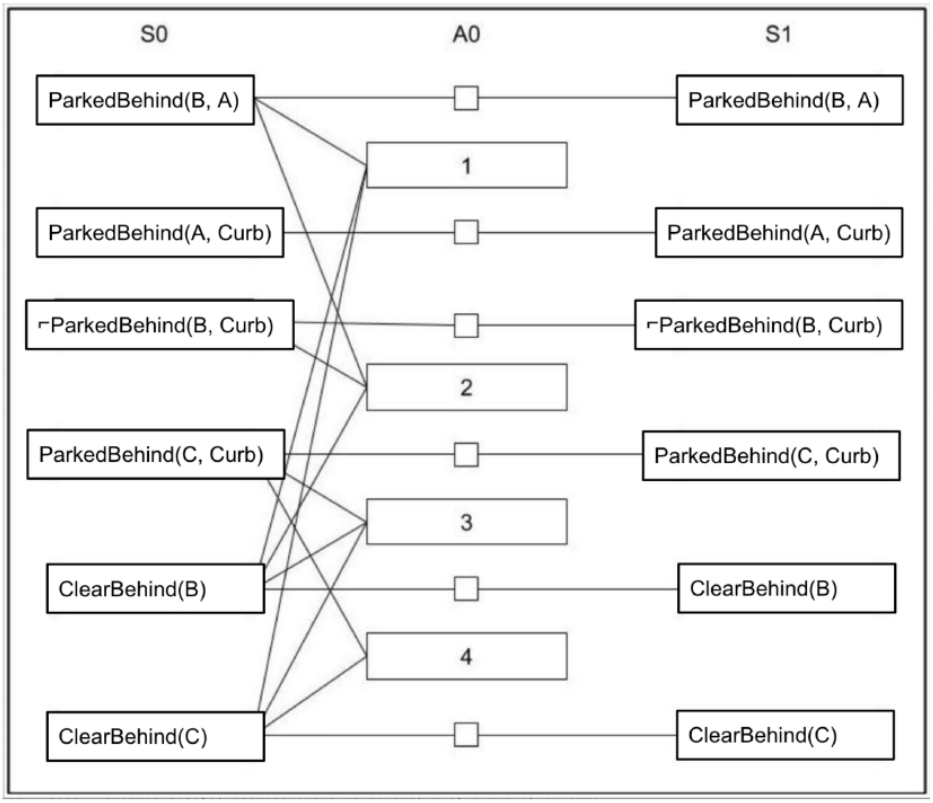
Câu c: Eat(Cheese) cần Has(Cheese), Gamble cần Rich. Ở S_2, Has(Cheese) chỉ được tạo bởi Buy(Cheese) ở A_1, còn Rich chỉ được giữ bởi No-op(Rich); hai hành động này mutex vì Buy(Cheese) thêm ¬Rich trong khi No-op(Rich) giữ Rich (Inconsistent Effects). Do đó Has(Cheese) và Rich mutex ở S_2 (inconsistent support), nên hai hành động có tiền điều kiện mutex, tức Competing Needs. Hai hành động này không có hiệu ứng mâu thuẫn và không xóa tiền điều kiện của nhau, nên lý do mutex duy nhất là Competing Needs.

Bài 4. Classical Planning và GraphPlan - Valet Parking

Xét bài toán Valet Parking với hai predicate ClearBehind(car) và ParkedBehind(car1, car2). Hai toán tử được định nghĩa như sau:

Thành phần	**ParkBehind(car1, car2)**	**ParkInNewRow(car)**
Preconditions	ClearBehind(car1) ClearBehind(car2) ParkedBehind(car1, place)	ClearBehind(car) ParkedBehind(car, place) ¬ParkedBehind(car, curb)
Add List	ParkedBehind(car1, car2) ClearBehind(place) ¬ParkedBehind(car1, place) ¬ClearBehind(car2)	ParkedBehind(car, curb) ClearBehind(place) ¬ParkedBehind(car, place)
Delete List	ClearBehind(car2) ParkedBehind(car1, place)	ParkedBehind(car, place) ¬ParkedBehind(car, curb)

Hình dưới là mẫu cho hai mức đầu tiên $S_0 \rightarrow A_0 \rightarrow S_1$. Các nút hành động No-op đã được biểu diễn bằng các ô vuông nhỏ không tên.



Mẫu GraphPlan cho Valet Parking

a) Điền tên hành động phù hợp vào các ô 1-4 ở mức A_0. Nếu không tồn tại hành động hợp lệ cho tập tiền điều kiện đó, ghi "N/A".

Ở S_0, xe A đỗ sát lề, xe B đỗ sau A, xe C đỗ sát lề; phía sau B và C đều trống. Đối chiếu các cạnh tiền điều kiện nối vào từng ô:

Ô	Tiền điều kiện nối vào ô	Hành động
1	ParkedBehind(B, A), ClearBehind(B), ClearBehind(C)	ParkBehind(B, C) (place = A)
2	ParkedBehind(B, A), ¬ParkedBehind(B, Curb), ClearBehind(B)	ParkInNewRow(B) (place = A)
3	ParkedBehind(C, Curb), ClearBehind(B), ClearBehind(C)	ParkBehind(C, B) (place = Curb)
4	ParkedBehind(C, Curb), ClearBehind(C)	N/A

Ô 4 ứng với ParkInNewRow(C), nhưng toán tử này cần ¬ParkedBehind(C, Curb) mà ở S_0 xe C đang đỗ sát lề (ParkedBehind(C, Curb)), nên không có hành động hợp lệ.

b) Với từng hành động 1-4, liệt kê TẤT CẢ predicate ở S_1 nhận add-edge từ hành động đó. Nếu hành động không hợp lệ, ghi "N/A".

Ô	Hành động	Predicate ở S_1 nhận add-edge
1	ParkBehind(B, C)	ParkedBehind(B, C), ClearBehind(A), ¬ParkedBehind(B, A), ¬ClearBehind(C)
2	ParkInNewRow(B)	ParkedBehind(B, Curb), ClearBehind(A), ¬ParkedBehind(B, A)
3	ParkBehind(C, B)	ParkedBehind(C, B), ClearBehind(Curb), ¬ParkedBehind(C, Curb), ¬ClearBehind(B)
4	N/A	N/A

Các literal này là literal mới xuất hiện ở S_1 (ngoài các literal được No-op mang từ S_0 sang). Với ô 3, place = Curb nên Add List sinh ra ClearBehind(Curb); literal này chỉ có ý nghĩa hình thức theo định nghĩa toán tử.

c) Nêu hai action node có quan hệ mutex do Inconsistent Effects. No-op được phép sử dụng.

Hành động 2 ParkInNewRow(B) và No-op của ¬ParkedBehind(B, Curb). Hành động 2 thêm ParkedBehind(B, Curb) (và xóa ¬ParkedBehind(B, Curb)) trong khi No-op giữ ¬ParkedBehind(B, Curb), hai hiệu ứng phủ định nhau.

d) Nêu hai action node có quan hệ mutex do Interference.

Hành động 1 ParkBehind(B, C) và hành động 3 ParkBehind(C, B). Hành động 1 xóa ClearBehind(C), là tiền điều kiện của hành động 3; ngược lại hành động 3 xóa ClearBehind(B), là tiền điều kiện của hành động 1.

e) GraphPlan "leveled off" khi nào? Chọn một đáp án:

☐ A. Tất cả các hành động có thể đã được khám phá.

☐ B. Không còn tập literal khác rỗng nào không có liên kết mutex.

☒ C. Hai mức liên tiếp của planning graph giống hệt nhau.

☐ D. Mức trạng thái cuối cùng chứa một trạng thái mục tiêu.

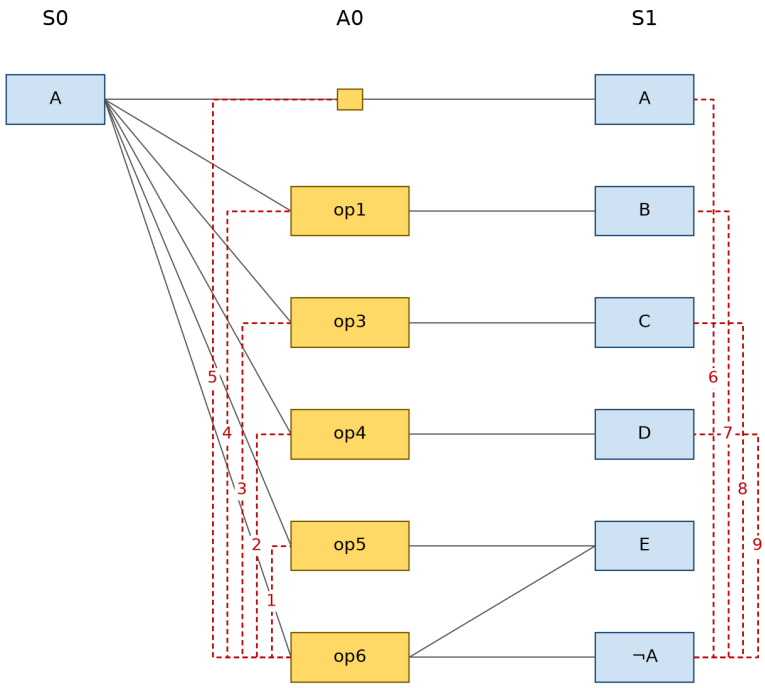
Đáp án C. Planning graph leveled off khi hai mức trạng thái liên tiếp có cùng tập literal và cùng các quan hệ mutex; từ đó mọi mức sau đều giống hệt, nên mở rộng thêm không tạo ra thông tin mới.

Bài 5. Xây dựng GraphPlan và trích xuất kế hoạch

Cho môi trường lập kế hoạch với 6 toán tử. Trạng thái bắt đầu là A; mục tiêu là C ∧ D ∧ E. Mỗi thời điểm chỉ áp dụng một toán tử và cần tìm kế hoạch có ít thao tác nhất.

	op1	**op2**	**op3**	**op4**	**op5**	**op6**
Precondition	A	B	A	A	A	A
Add	B	C, D, E	C	D	E	E, ¬A
Delete						

a) Chạy GraphPlan: vẽ lần lượt các mức trạng thái và mức hành động cho đến khi thuật toán dừng. Nhớ bổ sung các No-op cho trạng thái cần duy trì.



Khung để hoàn thiện planning graph; trạng thái đầu S_0 = {A}.

Ở S_0 chỉ có A. Mức A_0 gồm mọi toán tử có tiền điều kiện A là op1, op3, op4, op5, op6 cùng No-op(A); op2 chưa áp dụng được vì cần B. Mức S_1 = {A, B, C, D, E, ¬A}. Các literal mục tiêu C, D, E đều có ở S_1 và đôi một không mutex (C do op3, D do op4, E do op5; ba hành động này không mutex với nhau), nên GraphPlan thử trích xuất kế hoạch ngay ở mức này, thành công và dừng. Đường đứt đỏ đánh số là các cặp mutex, liệt kê ở câu d.

b) Kế hoạch nào được GraphPlan trả về?

GraphPlan trả về kế hoạch một mức: thực hiện op3, op4, op5 ở A_0 (theo thứ tự bất kỳ khi chỉ áp dụng một toán tử mỗi thời điểm, ví dụ op3 → op4 → op5). E phải lấy từ op5 chứ không từ op6, vì op6 mutex với op3 và op4.

c) Kế hoạch trên có tối ưu về số lượng thao tác hay không? ☐ Có ☒ Không

Không. Kế hoạch của GraphPlan dùng 3 thao tác, trong khi kế hoạch op1 → op2 chỉ dùng 2 thao tác (op1 tạo B, op2 tạo đồng thời C, D, E). GraphPlan tối ưu theo số mức (số bước song song): kế hoạch op3, op4, op5 chỉ cần 1 mức, còn op1 → op2 cần 2 mức, nên GraphPlan dừng ở mức 1 và không tìm kế hoạch ít thao tác hơn. Nếu ép mỗi mức chỉ có một hành động (mọi cặp hành động khác No-op đều mutex) thì GraphPlan sẽ tìm ra op1 → op2.

d) Liệt kê TẤT CẢ cặp toán tử mutex ở A_0 và TẤT CẢ cặp proposition mutex ở S_1. Nếu không có, ghi "Không có".

Số	Cặp mutex	Lý do
1	op6 – op5 (ở A_0)	Interference: op6 thêm ¬A, tức xóa A, là tiền điều kiện của op5
2	op6 – op4 (ở A_0)	Interference: op6 xóa A, tiền điều kiện của op4
3	op6 – op3 (ở A_0)	Interference: op6 xóa A, tiền điều kiện của op3
4	op6 – op1 (ở A_0)	Interference: op6 xóa A, tiền điều kiện của op1
5	op6 – No-op(A) (ở A_0)	Inconsistent Effects (và Interference): op6 thêm ¬A, No-op giữ A
6	A – ¬A (ở S_1)	Hai literal phủ định nhau
7	¬A – B (ở S_1)	¬A chỉ do op6, B chỉ do op1; op6 và op1 mutex
8	¬A – C (ở S_1)	¬A chỉ do op6, C chỉ do op3; op6 và op3 mutex
9	¬A – D (ở S_1)	¬A chỉ do op6, D chỉ do op4; op6 và op4 mutex

Các cặp còn lại không mutex. Ở A_0, op1, op3, op4, op5 và No-op(A) đôi một không mutex vì không xóa gì. Ở S_1, ¬A và E không mutex vì op6 tạo ra cả hai.

Bài 6. Tổng hợp và phân tích

a) So sánh tìm kiếm trực tiếp trong không gian trạng thái với GraphPlan. Trong trường hợp nào planning graph giúp giảm đáng kể không gian tìm kiếm?

Tìm kiếm trực tiếp (tiến hoặc lùi) duyệt từng trạng thái cụ thể; số trạng thái có thể tăng theo hàm mũ theo số literal, và cùng một tập hành động độc lập được thử theo mọi thứ tự khác nhau. GraphPlan không liệt kê trạng thái mà xây các mức gồm tập literal có thể đúng, kích thước tăng đa thức; quan hệ mutex loại sớm các tổ hợp không thể xảy ra, và việc trích xuất kế hoạch chỉ tìm kiếm lùi trong đồ thị đã thu hẹp. GraphPlan còn cho kế hoạch ngắn nhất theo số bước song song và phát hiện được bài toán vô nghiệm khi đồ thị leveled off.

Planning graph giúp giảm đáng kể không gian tìm kiếm khi có nhiều hành động độc lập có thể thực hiện song song (tìm kiếm trạng thái phải xét tới n! thứ tự cho n hành động như vậy, còn GraphPlan gom chúng vào một mức), khi mục tiêu gồm nhiều literal ít tương tác với nhau và kế

hoạch tương đối ngắn, hoặc khi cần nhanh chóng kết luận không có kế hoạch. Lợi ích giảm đi khi các hành động tương tác mạnh (rất nhiều mutex) và kế hoạch dài, phải thực hiện tuần tự.

b) Vì sao No-op quan trọng trong GraphPlan? Nếu bỏ toàn bộ No-op thì việc biểu diễn trạng thái tồn tại qua nhiều mức sẽ bị ảnh hưởng như thế nào?

No-op là hành động giả có tiền điều kiện p và hiệu ứng p, biểu diễn quán tính: một literal vẫn giữ nguyên nếu không có hành động nào thay đổi nó. Nhờ No-op, literal ở S_i được mang sang S_{i+1} nên các mức trạng thái tăng dần, đảm bảo điều kiện leveled off có ý nghĩa. Mutex giữa No-op và hành động thật cũng là cách GraphPlan thể hiện việc một hành động phá hủy literal đang được duy trì (Inconsistent Effects, Interference).

Nếu bỏ toàn bộ No-op, một literal chỉ có mặt ở đúng mức mà nó vừa được tạo ra và biến mất ở mức sau, trừ khi có hành động khác tạo lại. Khi đó không biểu diễn được các kế hoạch mà một điều kiện được tạo ra sớm nhưng dùng muộn, các literal mục tiêu đạt được ở những mức khác nhau không thể cùng có mặt ở một mức, các mức không còn đơn điệu nên không dùng được điều kiện leveled off, và GraphPlan sẽ bỏ sót kế hoạch hoặc kết luận sai là không có kế hoạch.

c) Cho một mục tiêu gồm nhiều literal. Hãy giải thích vai trò của mutex giữa các literal mục tiêu trong quyết định mở rộng planning graph hoặc kết luận thất bại.

GraphPlan chỉ thử trích xuất kế hoạch ở mức S_n khi mọi literal mục tiêu đều xuất hiện và không có cặp nào mutex với nhau. Nếu còn cặp literal mục tiêu mutex, chúng chưa thể cùng đúng sau n bước, nên GraphPlan mở rộng thêm một mức chứ không tốn công tìm kiếm. Vì mutex chỉ xét theo cặp, không mutex là điều kiện cần chứ chưa đủ: việc trích xuất vẫn có thể thất bại, khi đó tập mục tiêu thất bại ở mức đó được ghi nhớ (nogood) và đồ thị tiếp tục được mở rộng. Nếu đồ thị đã leveled off mà các literal mục tiêu vẫn còn mutex (hoặc chưa xuất hiện đủ), hoặc số nogood không thay đổi giữa hai lần mở rộng liên tiếp sau khi leveled off, GraphPlan kết luận bài toán không có kế hoạch.

d) Nêu quy trình tổng quát để trích xuất một kế hoạch từ planning graph sau khi tất cả literal mục tiêu đã xuất hiện ở cùng một mức và không mutex với nhau.

Bước 1: lấy tập mục tiêu G ở mức S_n. Bước 2: với mỗi literal trong G, chọn một hành động ở A_{n-1} có cạnh add tới literal đó (thường ưu tiên No-op), sao cho các hành động được chọn đôi một không mutex. Bước 3: hợp các tiền điều kiện của những hành động đã chọn trở thành tập mục tiêu mới ở mức S_{n-1}. Bước 4: lặp lại bước 2 và 3 lùi dần từng mức cho tới S_0; nếu tập mục tiêu ở S_0 nằm trong trạng thái đầu thì thành công. Bước 5: nếu ở một mức không chọn được tập hành động hợp lệ thì quay lui để thử lựa chọn khác; nếu mọi lựa chọn đều thất bại thì ghi nhớ (tập mục tiêu, mức) là nogood và mở rộng đồ thị thêm một mức rồi thử lại. Kế hoạch trả về là dãy các tập hành động A_0, A_1, …, A_{n-1} sau khi bỏ No-op; các hành động trong cùng một tập có thể thực hiện theo thứ tự bất kỳ.

— Hết —